



## Praktikum Penyelesaian Numerik

### Acara 6

# Persamaan Diferensial Parsial Partial Differential Equation

Theodorus Permana, Ph.D.

6 November 2024

# Definisi PDE

Sebuah persamaan yang menyelesaikan fungsi multivariabel yang terdiri dari satu atau lebih independent variable.

Ordinary differential equations can be viewed as a subclass of partial differential equations, corresponding to functions of a single variable.

## ODE:

$$\frac{d}{dt}x(t) = \lambda x(t)$$

hanya 1 variable independent,  
yaitu terhadap t.  
Contoh: gerak benda Newton,  
pertumbuhan populasi, peluruhan  
radioaktif

## PDE:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

terdapat 2 variable independent,  
yaitu terhadap t dan u.  
Contoh: propagasi gelombang,  
sebaran panas, distribusi dan  
sebaran fluida

<https://www.youtube.com/watch?v=ly4S0oi3Yz8>

# Linear Shallow Water Equation

## KEGUNAAN

Memodelkan perubahan kecil pada tinggi muka air (*water height*) dan kecepatan horizontal (*horizontal velocities*).

## ASUMSI

1. Lapisan tipis fluida (air) dengan densitas konstan
2. Panjang gelombang  $\gg$  kedalaman (tebal) badan air
3. Kecepatan horizontal  $\gg$  kecepatan vertikal

## CONTOH

Perambatan tsunami di lautan lepas (jauh dari pantai)

## REFERENSI

The Shallow Water Equations      Harvey  
Segur & Hiroki Yamamoto (2009)

[https://gfd.whoi.edu/wp-content/uploads/sites/18/2018/03/lecture8-harvey\\_136564.pdf](https://gfd.whoi.edu/wp-content/uploads/sites/18/2018/03/lecture8-harvey_136564.pdf)

# Linear Shallow Water Equation

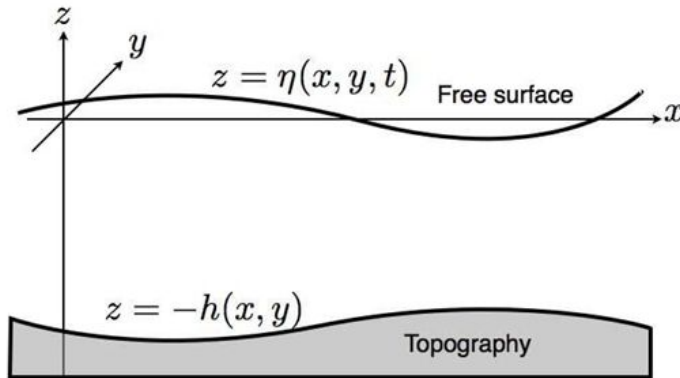


Figure 1: Schematic illustration of the Euler's system.

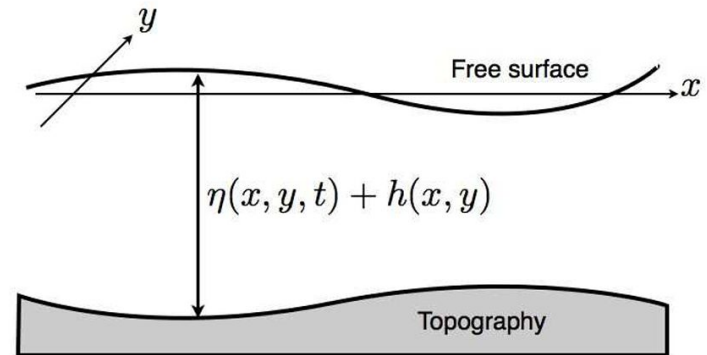


Figure 2: Schematic illustration of the shallow-water system.

## CONSERVATION OF MASS

$$\frac{\partial \eta}{\partial t} + \frac{\partial(uh)}{\partial x} + \frac{\partial(vh)}{\partial y} = 0$$

## HORIZONTAL MOMENTUM EQUATIONS

$$\frac{\partial u}{\partial t} + g \frac{\partial \eta}{\partial x} = 0$$

$u$ : x-component of velocity

$$\frac{\partial v}{\partial t} + g \frac{\partial \eta}{\partial y} = 0$$

$v$ : y-component of velocity

# Linear Shallow Water Equation

Dengan mengasumsikan topografi datar, maka tebal badan air  $h$  saat kondisi setimbang adalah konstan.

Jika  $h = 1$  m, maka:

## CONSERVATION OF MASS

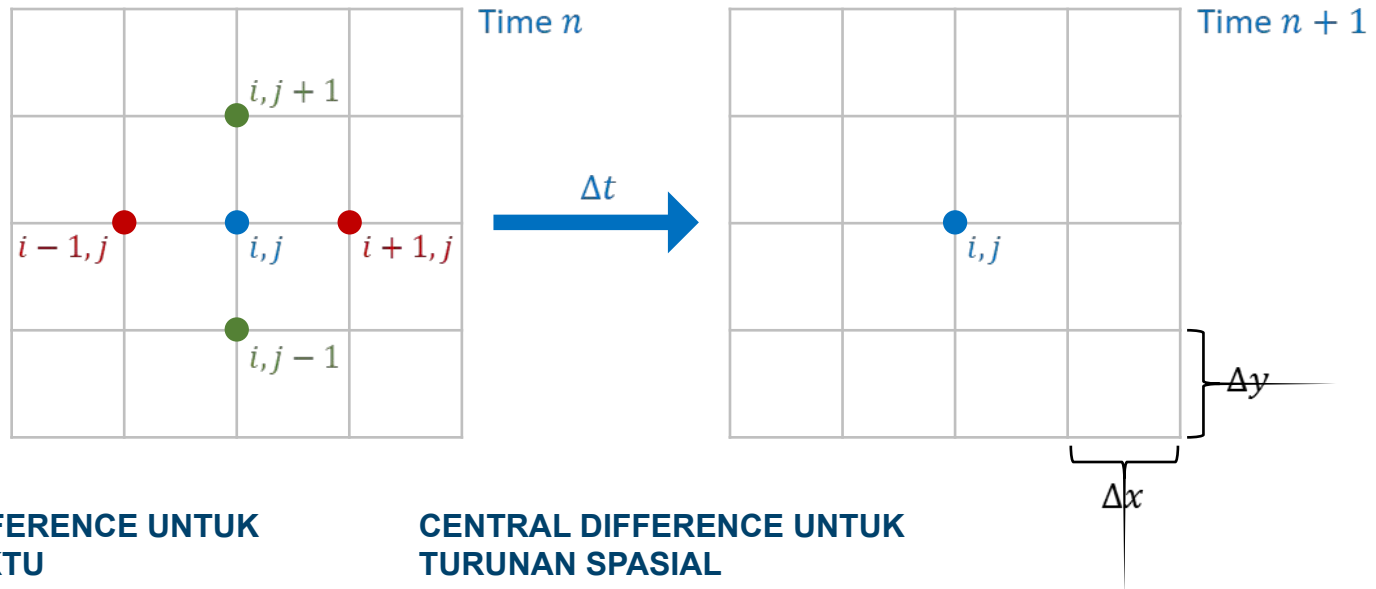
$$\frac{\partial \eta}{\partial t} + \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

## HORIZONTAL MOMENTUM EQUATIONS

$$\frac{\partial u}{\partial t} + g \frac{\partial \eta}{\partial x} = 0$$

$$\frac{\partial v}{\partial t} + g \frac{\partial \eta}{\partial y} = 0$$

# Finite Difference Approximation



**FORWARD DIFFERENCE UNTUK  
TURUNAN WAKTU**

$$\frac{\partial \eta}{\partial t} \rightarrow \frac{\eta_{i,j}^{n+1} - \eta_{i,j}^n}{\Delta t}$$

$$\frac{\partial u}{\partial t} \rightarrow \frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t}$$

$$\frac{\partial v}{\partial t} \rightarrow \frac{v_{i,j}^{n+1} - v_{i,j}^n}{\Delta t}$$

**CENTRAL DIFFERENCE UNTUK  
TURUNAN SPASIAL**

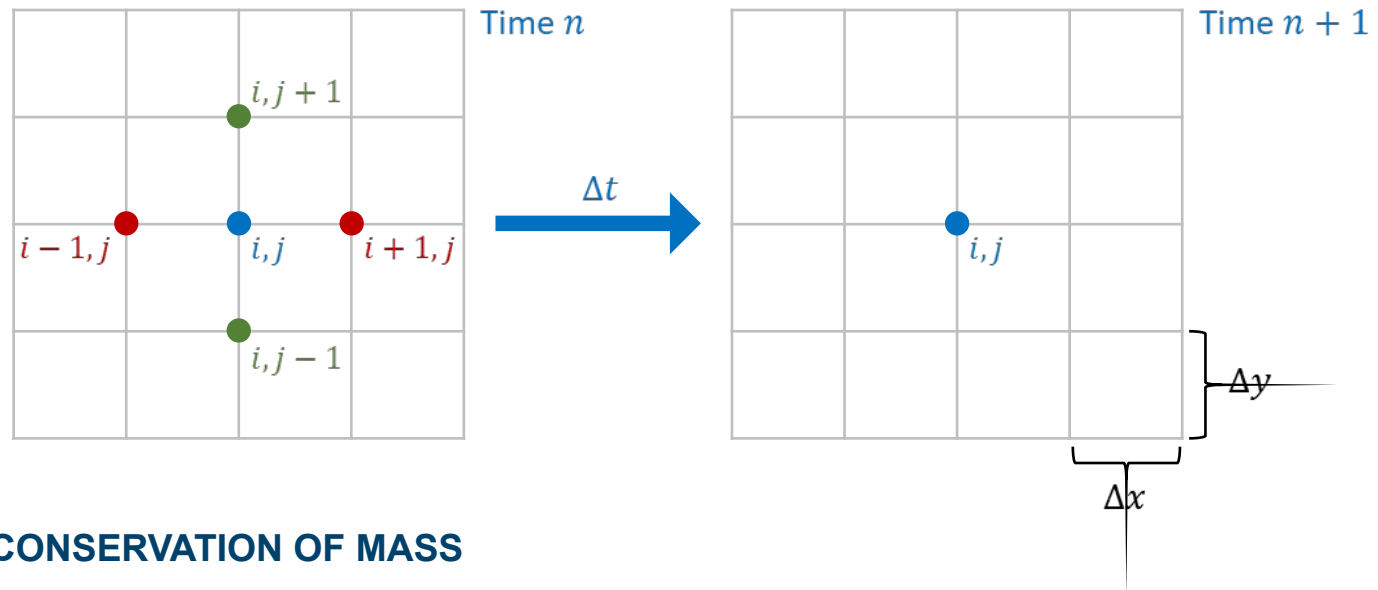
$$\frac{\partial \eta}{\partial x} \rightarrow \frac{\eta_{i+1,j}^n - \eta_{i-1,j}^n}{2\Delta x}$$

$$\frac{\partial u}{\partial x} \rightarrow \frac{u_{i+1,j}^n - u_{i-1,j}^n}{2\Delta x}$$

$$\frac{\partial v}{\partial y} \rightarrow \frac{v_{i,j+1}^n - v_{i,j-1}^n}{2\Delta y}$$

$$\frac{\partial \eta}{\partial y} \rightarrow \frac{\eta_{i,j+1}^n - \eta_{i,j-1}^n}{2\Delta y}$$

# Finite Difference Forms



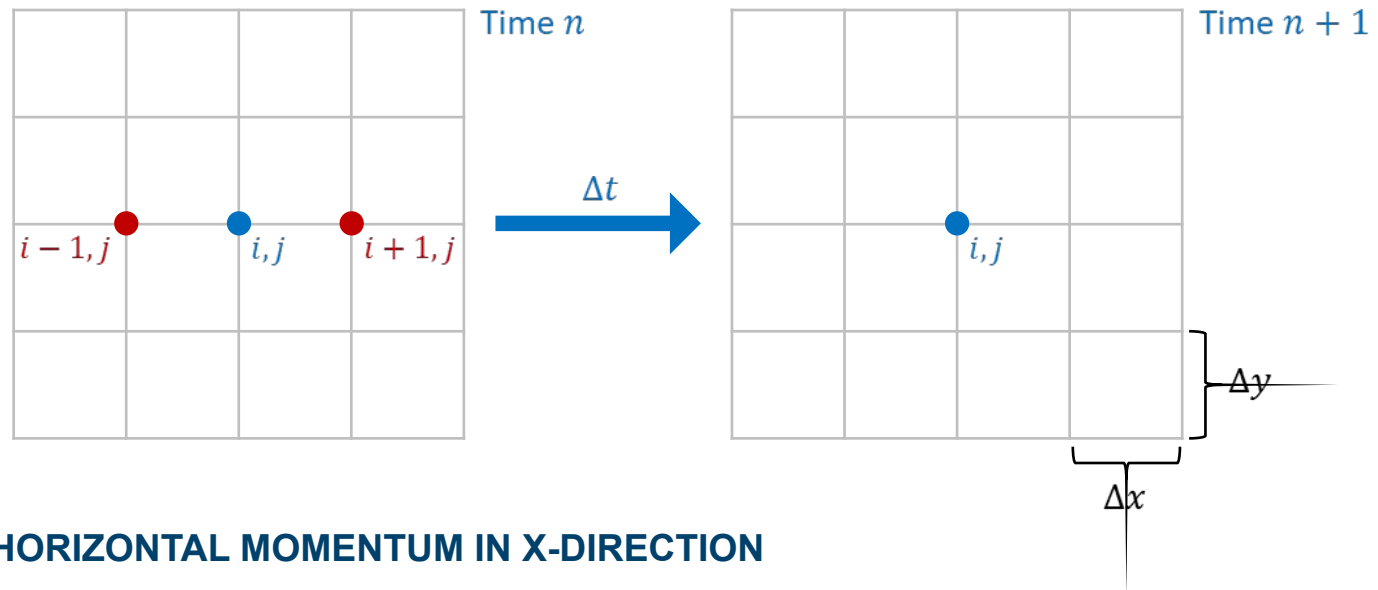
## CONSERVATION OF MASS

$$\frac{\partial \eta}{\partial t} + \frac{\partial (uh)}{\partial x} + \frac{\partial (vh)}{\partial y} = 0$$

$$\frac{\eta_{i,j}^{n+1} - \eta_{i,j}^n}{\Delta t} + \frac{u_{i+1,j}^n - u_{i-1,j}^n}{2\Delta x} + \frac{v_{i,j+1}^n - v_{i,j-1}^n}{2\Delta y} = 0$$

$$\eta_{i,j}^{n+1} = \eta_{i,j}^n - \Delta t \left( \frac{u_{i+1,j}^n - u_{i-1,j}^n}{2\Delta x} + \frac{v_{i,j+1}^n - v_{i,j-1}^n}{2\Delta y} \right)$$

# Finite Difference Forms



## HORIZONTAL MOMENTUM IN X-DIRECTION

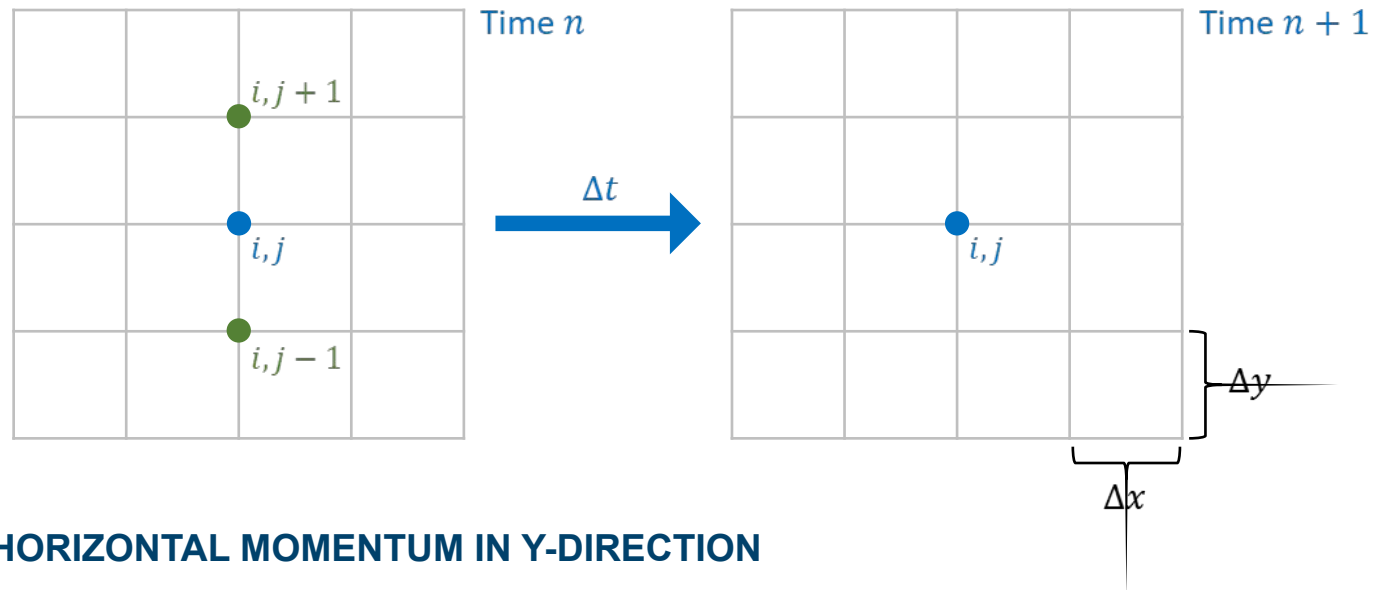
$$\frac{\partial u}{\partial t} + g \frac{\partial \eta}{\partial x} = 0$$

$$\frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t} + g \frac{\eta_{i+1,j}^n - \eta_{i-1,j}^n}{2\Delta x} = 0$$

$$u_{i,j}^{n+1} = u_{i,j}^n - g\Delta t \left( \frac{\eta_{i+1,j}^n - \eta_{i-1,j}^n}{2\Delta x} \right)$$



# Finite Difference Forms



## HORIZONTAL MOMENTUM IN Y-DIRECTION

$$\frac{\partial v}{\partial t} + g \frac{\partial \eta}{\partial y} = 0$$

$$\frac{v_{i,j}^{n+1} - v_{i,j}^n}{\Delta t} + g \frac{\eta_{i,j+1}^n - \eta_{i,j-1}^n}{2\Delta y} = 0$$

$$v_{i,j}^{n+1} = v_{i,j}^n - g\Delta t \left( \frac{\eta_{i,j+1}^n - \eta_{i,j-1}^n}{2\Delta y} \right)$$

# 1 | Initialization

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4
5 # Parameters
6 nx, ny = 100, 100      # Grid size
7 dx, dy = 0.5, 0.5      # Grid spacing (m)  $\Delta x$ ,
8 dt = 0.1               # Time interval (s)  $\Delta t$ 
9 g = 9.81               # Gravitational constant (m/s^2)  $g$ 
10 num_steps = 300        # Number of time steps
11
12 # Mesh grid for calculating gaussian function and plotting
13 x = np.linspace(0, nx * dx, nx)
14 y = np.linspace(0, ny * dy, ny)
15 X, Y = np.meshgrid(x, y)
16
17 # Initialize fields
18 # Water height at rest is assumed to be constant at 1 m
19 u = np.zeros((nx, ny)) # Velocity in x (m/s)  $u$ 
20 v = np.zeros((nx, ny)) # Velocity in y (m/s)  $v$ 
21
```

## 2 | Initial Condition

### 2D GAUSSIAN FUNCTION

$$G(x, y) = A \exp\left(-\frac{(x - x_0)^2}{2\sigma_x^2} - \frac{(y - y_0)^2}{2\sigma_y^2}\right)$$

$A$ : amplitude

$x_0, y_0$ : center position

$\sigma_x, \sigma_y$ : spread

$$\eta_{max} \rightarrow A$$

```

21
22 # Create an initial perturbation in the center of the grid
23
24 # maximum initial displacement
25 e_max = 1;
26
27 # Parameters for the 2D Gaussian
28 x0, y0 = nx * dx / 2, ny * dy / 2 # Center of the Gaussian
29 sigma_x, sigma_y = 2.0, 2.0 # Spread in x and y directions
30
31 # Define the 2D Gaussian function for the initial water displacement
32 e = e_max * np.exp(-((X - x0)**2 / (2 * sigma_x**2) + (Y - y0)**2 / (2 * sigma_y**2)))
33

```

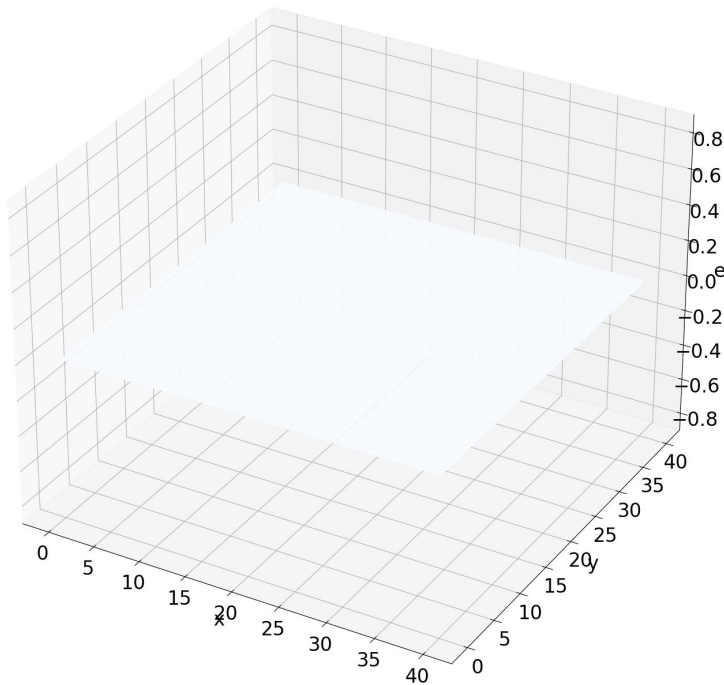
$\eta$

## INITIALIZATION

all-zero matrices

$u, v$

Hydrostatic equilibrium

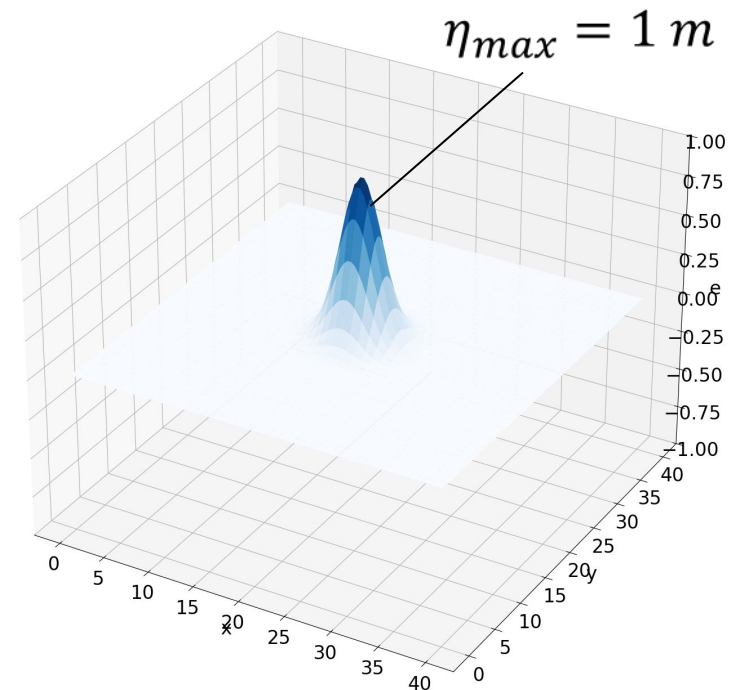


## INITIAL CONDITION

2D Gaussian function in the center

$\eta$

2D Gaussian function as initial displacement



### 3 | Prepare Visualization

```
33
34 # Set up the 3D plot
35 fig = plt.figure()
36 ax = fig.add_subplot(111, projection="3d")
37
38 # set ukuran font
39 plt.rcParams['font.size'] = 24
40
```

# 4 | Stability Condition

## COURANT-FRIEDRICHS-LEWY (CFL) CONDITION

For individual direction:

$$\Delta t \leq \frac{\Delta x}{\sqrt{g\eta_{max}}}$$

$$\Delta t \leq \frac{\Delta y}{\sqrt{g\eta_{max}}}$$

Or, in two-dimensional grid:

$$\Delta t \leq \min\left(\frac{\Delta x}{\sqrt{g\eta_{max}}}, \frac{\Delta y}{\sqrt{g\eta_{max}}}\right)$$

```
40
41 def check_stability(dt, dx, dy, g, e_max):
42
43     # Calculate the maximum allowed dt based on the CFL condition
44     max_dt_x = dx / np.sqrt(g * e_max)
45     max_dt_y = dy / np.sqrt(g * e_max)
46
47     # Compute the minimum allowed dt across both directions
48     max_dt = min(max_dt_x, max_dt_y)
49
50     # Check if the current time interval satisfies the CFL condition
51     return dt <= max_dt
52
```



# 5 | Conservation of Mass



IN FINITE-DIFFERENCE FORM

$$\eta_{i,j}^{n+1} = \eta_{i,j}^n - \Delta t \left( \frac{u_{i+1,j}^n - u_{i-1,j}^n}{2\Delta x} + \frac{v_{i,j+1}^n - v_{i,j-1}^n}{2\Delta y} \right)$$

```
52
53 # Update functions from the previous time step
54 def update_conservation_of_mass(e, u, v, dx, dy, dt):
55     e_new = np.copy(e)
56     e_new[1:-1, 1:-1] -= dt * (
57         (u[2:, 1:-1] - u[:-2, 1:-1]) / (2 * dx) +
58         (v[1:-1, 2:] - v[1:-1, :-2]) / (2 * dy)
59     )
60     return e_new
61
```



# 6 | Momentum Equations

## IN FINITE-DIFFERENCE FORM

$$u_{i,j}^{n+1} = u_{i,j}^n - g\Delta t \left( \frac{\eta_{i+1,j}^n - \eta_{i-1,j}^n}{2\Delta x} \right) \quad u: \text{x-component of velocity}$$

$$v_{i,j}^{n+1} = v_{i,j}^n - g\Delta t \left( \frac{\eta_{i,j+1}^n - \eta_{i,j-1}^n}{2\Delta y} \right) \quad v: \text{y-component of velocity}$$

```
61
62 def update_x_momentum(u, e, dx, dt, g):
63     u_new = np.copy(u)
64     u_new[1:-1, 1:-1] -= g * dt * (e[2:, 1:-1] - e[:-2, 1:-1]) / (2 * dx)
65     return u_new
66
67 def update_y_momentum(v, e, dy, dt, g):
68     v_new = np.copy(v)
69     v_new[1:-1, 1:-1] -= g * dt * (e[1:-1, 2:] - e[1:-1, :-2]) / (2 * dy)
70     return v_new
71
```



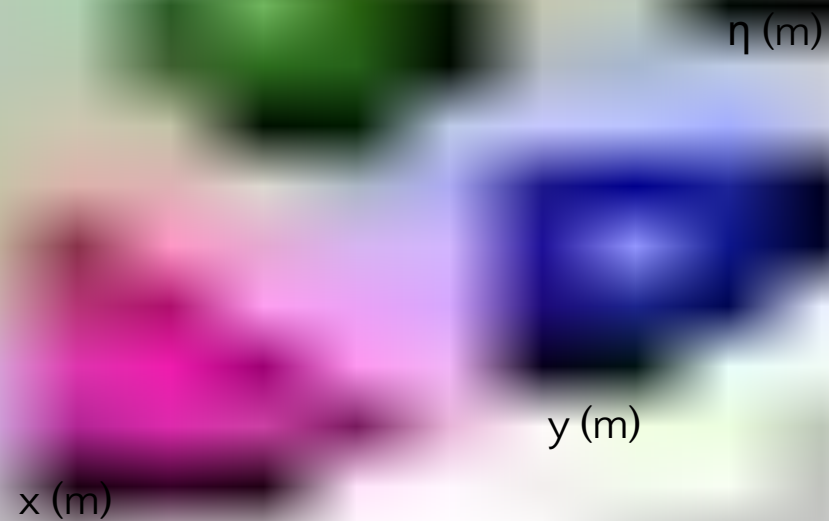
## 7 | Check for Stability

```
71
72 # Check for stability
73 cfl = check_stability(dt, dx, dy, g, e_max)
74
75 if cfl:
76     print('Kondisi stabilitas terpenuhi!')
77 else:
78     print('Simulasi tidak stabil!')
79
```

## 8 | Run Simulation

```
79
80 # Simulation loop
81 def simulate(frame):
82     global e, u, v
83
84     # Update fields
85     e = update_conservation_of_mass(e, u, v, dx, dy, dt)
86     u = update_x_momentum(u, e, dx, dt, g)
87     v = update_y_momentum(v, e, dy, dt, g)
88
89     # Clear the current plot
90     ax.clear()
91
92     # Plot updated surface
93     ax.plot_surface(X, Y, e, cmap="Blues", edgecolor="none")
94
95     # Set plot limits and labels
96     ax.set_zlim(-e_max, e_max)
97     ax.set_xlabel("x")
98     ax.set_ylabel("y")
99     ax.set_zlabel("e")
100     ax.set_title(f"Time {frame * dt:.3f} s (time step {frame})\n \Please write in
101                  Maximum height {1 + np.amax(e):.3f} m")          one line
102
103 # Create animation
104 ani = FuncAnimation(fig, simulate, frames=num_steps, repeat=False)
105
106 # Display the animation
107 plt.show()
108
```

## LINEAR SHALLOW WATER EQUATION



by Theodorus Permana  
6 November 2024

# You may bring your Python to rest



UNIVERSITAS  
GADJAH MADA

LOCALLY ROOTED, GLOBALLY RESPECTED

[ugm.ac.id](http://ugm.ac.id)